



FACHBEREICH INFORMATIK

MASTER'S THESIS

CREATING AND BENCHMARKING LEARNED DATABASE COMPONENTS USING LLMs

Author:
Preksha Rajesh Shah

submitted on:
April 29, 2026

Supervisor
Angjela Davitkova

Reviewers
Prof. Dr.-Ing. Sebastian Michel

Abstract

Learned cardinality estimation has shown that neural networks can outperform traditional rule-based estimators, but training these models requires large volumes of labeled data obtained through expensive database query execution. This thesis presents a framework that combines large language model-based query generation with active learning to reduce this labeling cost. An LLM generates diverse SQL workloads from a given database schema, a four-layer validation pipeline filters invalid queries, and an active learning loop selects only the most informative queries for execution and labeling. The trained Multi-Set Convolutional Network achieves competitive estimation accuracy while requiring significantly fewer labeled queries than a fully supervised baseline. The entire pipeline is schema-agnostic and fully automated, enabling scalable training of learned database components on arbitrary relational schemas.

Zusammenfassung

Gelernte Kardinalitätsschätzung hat gezeigt, dass neuronale Netze traditionelle regelbasierte Schätzer übertreffen können. Das Training solcher Modelle erfordert jedoch große Mengen gelabelter Daten, die durch teure Datenbankabfragen gewonnen werden müssen. Diese Arbeit stellt ein Framework vor, das LLM-basierte Anfragegenerierung mit aktivem Lernen kombiniert, um diese Kosten zu reduzieren. Ein LLM erzeugt diverse SQL-Workloads aus einem gegebenen Datenbankschema, eine vierstufige Validierungspipeline filtert ungültige Anfragen, und eine Active-Learning-Schleife wählt gezielt die informativsten Anfragen zur Ausführung und Labelung aus. Das trainierte Multi-Set Convolutional Network erreicht wettbewerbsfähige Schätzgenauigkeit bei deutlich weniger gelabelten Anfragen als ein vollständig überwachter Ansatz. Die gesamte Pipeline ist schemaagnostisch und vollautomatisiert, was ein skalierbares Training gelernter Datenbankkomponenten auf beliebigen relationalen Schemata ermöglicht.

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Proposed Approach	1
1.3	Contributions	2
2	Background	3
2.1	Learned Database Systems and Cardinality Estimation	3
2.2	MSCN and Deep Learning-based Estimation	3
2.3	Workload Sensitivity and Challenges	4
2.4	LLM-based SQL Generation	5
2.5	Generative AI for Database Components	5
2.6	Uncertainty Quantification via Monte Carlo Dropout	6
2.7	Active Learning for Data Efficiency	7
2.8	Data Diversity, Personas, and KL Divergence	7
2.9	Local Model Hosting for Reproducibility	7
3	Related Work	9
3.1	Learned Cardinality Estimation	9
3.2	Workload Characteristics and Model Generalization	9
3.3	LLM-based SQL Workload Generation	10
3.4	Generative AI for Training Database Components	10
3.5	Active Learning and Uncertainty Quantification	11
3.6	Data Diversity and Distribution Alignment	12
3.7	Positioning of the Proposed Approach	12
4	Approach	13
4.1	System Architecture	13
4.2	Pipeline Walkthrough	13
4.3	Why This Framework	14
5	Query Generation Approach	16
5.1	Why Use a Language Model	16
5.2	Prompt Design	16
5.3	Keeping the Workload Diverse	18
5.4	Filtering Out Bad Queries	18
5.5	Synthetic Fallback	19
6	Active Learning	21
6.1	From Random Labeling to Intelligent Selection	21
6.2	Acquisition Strategies	22
6.3	The Training Loop	23
6.4	Why It Works for Cardinality Estimation	24
7	Training Approach	26
7.1	Database Labeling	26
7.2	Feature Extraction and Bitmap Generation	26
7.3	Multi-Set Convolutional Network	27

7.4	Normalization and Loss	29
8	Experimental Evaluation	30
8.1	Experimental Setup	30
8.2	Query Generation Analysis	30
8.3	Active Learning and Model Training Results	33
8.4	Labeling Efficiency Analysis	37
8.5	Comparison with PostgreSQL Optimizer	37
8.6	Discussion	38
9	Conclusion	42
9.1	What We Built and What We Learned	42
9.2	Limitations	42
9.3	Directions for Future Work	43

List of Figures

4.1	End-to-end pipeline from schema ingestion through query generation, validation, labeling, and active learning to final model evaluation. The dashed feedback loop represents the iterative active learning cycle.	13
5.1	Five-part prompt template. The upper components establish what the model knows; the lower components constrain what it is allowed to produce.	17
5.2	Four-layer validation pipeline. Cheap structural checks run first, expensive schema lookups run last. Queries rejected at any stage are logged for post-hoc analysis.	20
7.1	MSCN architecture. Three independent neural modules process tables, predicates, and joins. Their pooled representations are concatenated and mapped to a cardinality estimate in $[0, 1]$	28
8.1	Entity-relationship diagram of the IMDB database schema. The <code>title_principals</code> table acts as a central junction connecting titles to people, while <code>title_ratings</code> provides aggregate user feedback per title. These relationships define the valid join paths for generated queries.	31
8.2	Structural feature distributions across the generated workload. The balanced coverage of different table counts, join complexities, and predicate counts confirms that the diversity hint mechanism prevents the LLM from converging on a narrow subset of query patterns.	32
8.3	Query validation success rates across the four filtering layers. The high pass rate at each stage demonstrates the effectiveness of the prompt engineering constraints in guiding the LLM toward valid query structures.	32
8.4	Distribution of table count per query. The weighted mix of single-table, two-table, and three-table queries ensures coverage of both simple selections and complex multi-way joins.	33
8.5	Distribution of predicate count per query. The majority of queries contain one to three predicates, reflecting a realistic range of filter complexities.	33
8.6	Distribution of join conditions per query. Queries with zero joins correspond to single-table queries, while one and two join conditions correspond to two-table and three-table queries, respectively.	34
8.7	Distribution of true cardinalities ($\log_{10}$ scale). The wide range spanning several orders of magnitude forces the model to learn accurate estimates for both highly selective queries returning few rows and broad queries spanning millions of tuples.	34

8.8	Active learning curve showing validation median Q-error versus the number of labeled queries. The steep initial decline demonstrates rapid learning from the first few rounds, while the flattening tail indicates diminishing returns as the model approaches its capacity on the given data.	35
8.9	Training loss curves across active learning rounds. The consistently decreasing trajectory demonstrates stable convergence, with each round starting from a better baseline than the previous one due to the expanded and more informative training set. . . .	35
8.10	Predicted vs. actual cardinality on a log-log scale. The concentration of points along the diagonal indicates that the MSCN model produces well-calibrated estimates across the full cardinality range, from selective single-table queries to large multi-table joins.	36
8.11	Cumulative distribution function of Q-errors. The steep rise at low Q-error values indicates that the majority of queries are estimated within a factor of 2–5 of their true cardinality.	37
8.12	Q-error box plots per active learning round. The narrowing boxes and decreasing medians demonstrate how each round improves estimation accuracy.	37
8.13	Q-error statistics across active learning rounds. The consistent decrease in all percentile metrics confirms that active query selection improves not only typical-case accuracy but also tail-end performance on the most challenging queries.	38
8.14	Labeling efficiency comparison. The active learning approach achieves competitive accuracy with significantly fewer labeled queries than a fully supervised baseline, demonstrating that intelligent query selection reduces the dominant cost factor in the pipeline.	39
8.15	Labeling success rate across the pipeline. Queries that exceed the database timeout are assigned fallback cardinalities. The high success rate confirms that the generated queries are overwhelmingly executable within the configured time limits.	40
8.16	Q-error CDF comparing the MSCN model against PostgreSQL’s native estimator. A curve shifted further left indicates better estimation accuracy.	40
8.17	Predicted vs. actual scatter comparing MSCN and PostgreSQL. Points closer to the diagonal represent more accurate estimates. .	40
8.18	Pipeline summary dashboard. Each panel corresponds to a pipeline stage, making it possible to diagnose bottlenecks and quality issues at a glance.	41

List of Algorithms

1	LLM-Based Query Generation	18
2	Active Learning for Cardinality Estimation	23

List of Tables

5.1	LLM-based versus synthetic query generation.	19
8.1	Experimental configuration parameters.	31

Chapter 1

Introduction

1.1 Problem Statement

Query optimization in relational databases hinges on cardinality estimation predicting how many rows a query will return before actually executing it. Get the estimate right and the optimizer picks a fast execution plan. Get it wrong and the plan can be orders of magnitude slower than necessary. Traditional estimators lean on assumptions like attribute independence and uniform value distributions that rarely hold in practice, and the resulting errors compound across joins, leading to plans that are far from optimal.

Over the past few years, learned cardinality estimators have shown that neural networks can do a better job. The Multi-Set Convolutional Network (MSCN) by Kipf et al. [6] demonstrated that a relatively simple deep learning model can capture cross-table correlations that rule-based estimators miss entirely. But learned models come with their own cost: they need large quantities of labeled training data, and each label requires executing a query on a live database to record its true row count. On a complex schema this execution-based labeling can take hours, making it the dominant bottleneck in any pipeline that trains learned estimators from scratch.

Meanwhile, large language models have become remarkably good at generating SQL. Work like SQLStorm [13] has shown that LLMs can produce diverse, realistic query workloads at scale when given a schema and a well-crafted prompt. But generating queries is only half the problem. Those queries still need to be executed to obtain labels, and that execution cost remains.

The question this thesis addresses is: can we close the loop generate a large, diverse pool of queries with an LLM, then use active learning to pick only the most informative ones for labeling so that we get a good estimator without paying the full cost of executing every query in the pool?

1.2 Proposed Approach

We treat the LLM not as a runtime component but as a data factory. Given a database schema, the LLM generates thousands of synthetic `SELECT COUNT(*)` queries. These queries are validated, filtered, and encoded into the tensor format that MSCN expects. Then, instead of labeling them all, we use active learning to identify the queries that the model is most uncertain about and label only those. The model trains on the labeled subset, its uncertainty shifts, and the next round of selection targets different queries. After several rounds the model has seen a compact but highly informative training set.

The whole process is packaged as an automated, schema-agnostic pipeline. A researcher points it at a database, specifies a few hyperparameters, and walks

away. The pipeline generates queries, validates them through four filtering layers, labels the selected subset, trains the MSCN model, and outputs a full evaluation dashboard. No manual query writing, no template engineering, no human in the loop.

The motivation draws on two bodies of prior work. Wang et al. [14] showed that the diversity and structural properties of the training workload have a profound effect on learned estimator quality, and Davitkova and Michel [2] demonstrated that generative AI can automate the creation of training data for database components. We build on both: using LLMs for diverse workload generation and active learning for efficient labeling.

1.3 Contributions

This thesis makes four contributions. First, it demonstrates that LLMs can serve as effective workload generators for learned cardinality estimation, producing queries with structural diversity that rivals or exceeds what template-based methods achieve. Second, it integrates active learning into the training loop, showing that uncertainty-based query selection reduces the labeling budget needed to reach a given accuracy level. Third, it packages everything into a fully automated pipeline that works out of the box on any schema with defined foreign keys, lowering the barrier to entry for researchers who want to experiment with learned estimators. Fourth, it provides an extensive experimental evaluation on the IMDB benchmark that quantifies the trade-offs between different acquisition strategies, workload sizes, and training configurations.

Chapter 2

Background

2.1 Learned Database Systems and Cardinality Estimation

Modern database systems use query optimizers to optimize queries for efficient execution. At the heart of query optimization is cardinality estimation. Cardinality estimation is used to predict the number of tuples that are generated during intermediate steps of query execution. The estimates are used to optimize queries and improve performance. However, small errors in cardinality estimation can lead to poor query optimization and performance. Conventional cardinality estimation techniques use statistical estimates such as histograms and sampling. Other techniques include making assumptions such as attribute independence and uniform distribution of values. Although conventional techniques are computationally efficient, they do not reflect real-world data characteristics. Real-world data has complex correlations among attributes and tables, especially for join-heavy queries. As such, conventional techniques often yield poor estimates for queries with multiple predicates and joins.

In addition to inaccuracies caused by independence assumptions, another limitation of traditional approaches lies in their inability to adapt dynamically to evolving data distributions. In real-world applications, data is continuously updated, and query workloads may change over time. Static statistical summaries often fail to capture these changes effectively, leading to outdated estimates and suboptimal execution plans. This problem becomes more severe in large-scale systems where maintaining accurate statistics is itself a costly operation. As a result, there has been a growing interest in approaches that can learn directly from data and adapt to changing workloads.

This limitation has motivated the exploration of learned database systems, where machine learning models replace traditional components. Instead of relying on predefined assumptions, these models learn patterns directly from data, enabling them to capture correlations that are otherwise difficult to model. Learned cardinality estimation is one of the most actively studied areas within this paradigm. These models aim to generalize across different query structures and data distributions, providing more robust and accurate estimates compared to traditional techniques.

2.2 MSCN and Deep Learning-based Estimation

One important advancement in the area of learning-based query cardinality estimation is the development of a new technique called Multi-Set Convolutional Network (MSCN), proposed by Kipf et al. [6]. The technique reformulates query cardinality estimation as a supervised learning task, where query result sizes are

estimated based on query structures.

The basic idea behind the MSCN technique is that it represents queries in the form of sets. Unlike traditional methods, where a query is simply represented as a string of characters, MSCN represents a query as three different sets, namely tables, joins, and predicates. Each set is processed separately using shared layers of a neural network. The basic idea behind this is to make the query invariant to the ordering of query components, which is important because the semantics of a query do not depend on the order in which tables and predicates are listed.

The other key feature of the MSCN is its use of bitmap-based sampling. In the process, a subset of the tuples in a given base table is sampled, and the predicates are applied to the sample to produce bitmaps. The bitmaps then represent the query predicates that are satisfied, and this additional information helps the model to better account for the correlations between the predicates and the data.

In addition, the MSCN uses normalization to ensure that the cardinality estimates are within a certain range that can be learned by the model. This is crucial, especially considering that the cardinality estimates can differ considerably across queries. The MSCN uses loss functions that involve relative errors, as opposed to absolute errors, which is more appropriate for query optimization tasks.

Despite its strong performance, MSCN depends heavily on labeled training data. Each training example requires a query and its true cardinality, which is obtained by executing the query on a database. As query complexity increases, the cost of execution grows significantly, making it difficult to scale the training process. This limitation highlights a fundamental challenge in learned database systems: the high cost of data generation. In addition, the reliance on execution makes it difficult to quickly generate new training data when schemas or workloads change.

2.3 Workload Sensitivity and Challenges

The performance of learned cardinality estimators not only depends on the architecture used but also on the nature and characteristics of the training workload. The study conducted by Wang et al. [14] offers a comprehensive analysis on how different query properties influence the performance of learned cardinality estimators.

For example, a learned model may not perform well when it is tested on queries that have different types of predicates, such as range predicates, when it is only trained on queries that have equality predicates. Similarly, a query may have high selectivity and join complexity, causing a large estimation error. This example again emphasizes that the training data should be highly diverse.

Another important aspect to be considered is related to workload shift, where the query distribution is different from the training distribution. In this case, learned models may not perform well, and this again emphasizes that the training data should be highly diverse. Moreover, the interaction between multiple

query features, such as the number of joins and predicate selectivity, can further complicate the estimation process. These interactions are often difficult to model explicitly, which reinforces the importance of exposing models to a wide variety of query patterns during training.

2.4 LLM-based SQL Generation

Recently, large language models have been recognized as effective means for generating structured data. In this regard, the SQLStorm framework [13] has been presented to show that LLMs can be effectively used to produce large volumes of varied and realistic SQL queries. In this regard, the SQLStorm framework exploits the capabilities of schema-informed prompting to produce varied and realistic queries.

Unlike other query generation techniques that are based on template-based and rule-based approaches, LLMs can be effectively used to produce queries that can capture a wide range of query patterns. This is one of the main reasons that LLMs can be effectively used to produce training datasets for learning-based query generation techniques. In addition to this, LLMs can be effectively used to produce queries that are not included in existing workloads. This allows for the exploration of new query patterns.

Another important advantage of LLM-based generation is its ability to incorporate semantic understanding of schemas. By conditioning on table and column descriptions, LLMs can generate queries that are not only syntactically correct but also semantically meaningful. This capability is particularly useful when dealing with complex schemas, where manual query generation becomes challenging.

Although LLMs can be effectively used to produce queries, they do not have the capability to produce the labels that are associated with the queries. This means that the produced queries have to be executed to produce labels. This is one of the main drawbacks of using LLMs for query generation, as it reintroduces the dependency on database execution.

2.5 Generative AI for Database Components

Recent foundational work by Davitkova and Michel [2] attempts to address this data generation bottleneck through a framework that utilizes generative models to automatically generate training data for learned database components. The basic idea is to utilize LLMs to automatically generate realistic SQL queries, thereby reducing reliance on manually collected query logs. This is a major departure from the traditional way learned database systems have been designed. By automating query generation, it is possible to generate large and varied datasets to support a wide range of query types. The framework also outlines how generative models can be incorporated into the learning pipeline to support data generation.

An important contribution of this work is the structured pipeline it proposes, where query generation, validation, execution, and model training are integrated

into a unified framework. This pipeline ensures that generated queries are not only diverse but also executable and meaningful within the given schema. The use of prompt engineering and iterative refinement further improves the quality of generated queries.

One limitation of this framework is that it relies on executing the generated queries to obtain ground truth labels. While query generation is now automated, label generation is computationally intensive and thus limits scalability. This is where a completely synthetic method is proposed, where query and label generation can be performed without relying on database execution. This shift represents a natural evolution of the framework, aiming to remove the remaining bottleneck in the data generation process.

2.6 Uncertainty Quantification via Monte Carlo Dropout

The ability to quantify the uncertainty of a machine learning model’s predictions is crucial for deciding whether a given estimate can be trusted. In the context of neural networks, one of the most effective techniques for uncertainty quantification is Monte Carlo (MC) dropout. To understand MC dropout, it is first necessary to examine the theoretical foundations of standard dropout.

Dropout was originally introduced as a regularization technique to prevent overfitting. As formally analyzed by Baldi and Sadowski [1], dropout forces neurons to become robust and rely on population behavior rather than co-adapting with specific units. Their theoretical framework demonstrates that applying dropout to a neural network essentially performs a geometric averaging over an ensemble of all possible subnetworks. By stochastically dropping units during training, the network is trained as a massive ensemble, and during standard inference, weight scaling approximates this ensemble average.

While standard dropout is turned off during inference, Gal and Ghahramani [4] demonstrated that keeping dropout active during prediction allows the network to function as a Bayesian approximation. By running multiple forward passes for the same input with different dropout masks, the network produces a distribution of predictions. The variance across these predictions serves as a principled measure of the model’s epistemic uncertainty.

However, a major limitation of MC dropout is its computational cost. Running dozens of forward passes for every query during inference introduces significant overhead, which has traditionally limited its use in performance-critical applications. Recent advancements have sought to address this bottleneck. For instance, Srinivasan [3] introduced a model-splitting technique that significantly accelerates MC dropout inference. By recognizing that dropout is often only applied to the final layers (the model head), the deterministic lower layers (the model body) only need to be executed once. The intermediate activations are then passed repeatedly through the stochastic head, yielding a potential 25x to 33x speedup. This makes MC dropout practically viable for rapid, millisecond-level uncertainty quantification, enabling its use in dynamic environments like active learning for database query optimization.

2.7 Active Learning for Data Efficiency

Active learning helps in developing a framework for improving data efficiency, which can be done through the selection of the best data samples for learning. As discussed by Li et al. [7], not all data samples are equally important in improving the performance of the model. This way, it is possible to attain high accuracy with less data.

In the context of LQE using learned cardinality estimation, active learning helps in improving the efficiency of the process of query labeling. Instead of labeling all the generated queries, the model can be improved through the selection of the best queries in improving the performance of the model. This selection of the best queries can be done using the uncertainty of the model.

In addition to uncertainty-based selection, active learning can also incorporate diversity-based sampling, where queries that represent different regions of the query space are selected. This ensures that the model is exposed to a wide range of patterns while avoiding redundancy in the training data. Such strategies are particularly useful when working with large volumes of synthetically generated data.

The integration of active learning with the process of synthetic data generation helps in improving the efficiency of the learning process. LLMs are capable of generating a wide variety of data samples in the form of queries. Active learning helps in the selection of the best data samples, thus improving the efficiency of the learning process.

2.8 Data Diversity, Personas, and KL Divergence

Diversity in the generated data is a critical factor to be taken into consideration for the training of strong models. The PAARS framework [9] proposes the use of personas for generating diverse and realistic data. Personas help in the simulation of diverse behavior, which enables the generation of diverse data.

In order to test the quality of the generated data, the PAARS framework uses the KL divergence as a similarity metric between the generated data and real-world data. The KL divergence is a measure of the dissimilarity between two probability distributions, which can be used to test the similarity between the generated data and real-world data, especially in the case of SQL queries.

In the context of query generation, KL divergence can be applied to distributions over query features such as the number of joins, predicate types, and selectivity ranges. By comparing these distributions between synthetic and real workloads, it is possible to evaluate whether the generated data accurately reflects real-world patterns. This provides a quantitative basis for improving the quality of synthetic datasets.

2.9 Local Model Hosting for Reproducibility

The practical deployment of LLMs for data generation requires careful consideration of reproducibility, privacy, and cost. In this work, we adopt a local hosting

strategy using the Ollama framework, which enables running open-source language models entirely on the researcher’s own hardware. This choice offers several important benefits for scientific experimentation. Local hosting eliminates dependency on external API services, ensuring that experiments are not affected by changes in third-party model versions, pricing, or availability. It also preserves data privacy, as database schemas and query workloads never leave the local environment. Furthermore, local inference provides consistent latency characteristics and enables full control over model parameters such as temperature and token limits, supporting reproducible experimental conditions.

Chapter 3

Related Work

3.1 Learned Cardinality Estimation

The use of machine learning to replace traditional database components has gained significant attention, particularly in the context of cardinality estimation. A foundational contribution in this area is the Multi-Set Convolutional Network (MSCN) proposed by Kipf et al. [6], which reformulates cardinality estimation as a supervised learning problem. Unlike conventional estimators that rely on simplifying assumptions such as attribute independence, MSCN represents queries as sets of tables, joins, and predicates, allowing the model to capture complex correlations across query components.

A key strength of MSCN lies in its ability to model join-crossing correlations, which are difficult for traditional estimators to handle effectively. Furthermore, the use of bitmap-based sampling enables the model to incorporate information about intermediate query results, improving estimation accuracy for complex queries involving multiple joins and predicates.

Despite these advantages, MSCN fundamentally depends on execution-based labeling. Each training instance requires executing a query on a database to obtain its true cardinality. This introduces substantial computational overhead and limits scalability, particularly when large and diverse workloads are required. Moreover, adapting to new schemas or workloads necessitates additional query execution, further increasing the cost. These limitations highlight a central challenge in learned cardinality estimation: the dependency on labeled data generated through expensive query execution.

3.2 Workload Characteristics and Model Generalization

Beyond model architecture, the effectiveness of learned cardinality estimators is strongly influenced by the characteristics of the training workload. Wang et al. [14] provide a comprehensive analysis demonstrating that learned models are highly sensitive to factors such as predicate types, selectivity, join complexity, and query structure.

A key observation is that models trained on narrow or biased workloads often fail to generalize to unseen queries. For example, a model trained primarily on equality predicates may perform poorly when evaluated on range predicates or more complex join conditions. Similarly, variations in selectivity can significantly impact estimation accuracy, as models struggle to extrapolate beyond observed distributions.

The issue of workload shift further complicates deployment. In real-world sys-

tems, query distributions evolve over time, leading to discrepancies between training and inference workloads. Under such conditions, learned models can exhibit significant performance degradation. These findings emphasize the importance of constructing training datasets that are both diverse and representative of real-world query patterns.

Recent work on robust cardinality estimation under changing workloads [10] further highlights the importance of adaptability, reinforcing the need for data generation approaches that can capture a wide range of query characteristics.

3.3 LLM-based SQL Workload Generation

The emergence of large language models (LLMs) has introduced new possibilities for SQL workload generation. The SQLStorm framework [13] demonstrates that LLMs can generate large-scale, diverse, and realistic SQL queries using schema-aware prompting techniques.

Unlike traditional template-based approaches, which are inherently limited in diversity, LLM-based methods can generate queries with varying levels of complexity, including different join structures, predicate combinations, and nested queries. This flexibility enables the generation of workloads that better reflect real-world usage patterns.

Additional work in text-to-SQL and SQL understanding, such as CHASE-SQL [11] and evaluation studies by Rahaman et al. [12], further demonstrate the capability of LLMs to understand and generate structured queries. Similarly, TAPEX [8] explores pre-training techniques for table reasoning, highlighting the broader applicability of neural models in structured data tasks.

However, existing LLM-based generation approaches primarily focus on query generation rather than labeled data creation. Generated queries still require execution to obtain ground truth cardinalities, thereby retaining the dependency on database systems and limiting scalability for training purposes.

3.4 Generative AI for Training Database Components

Recent work on integrating generative AI into database systems has sought to address the limitations of manual and execution-based data generation. Davitkova and Michel [2] propose a framework that leverages generative models to automate the training of learned database components.

This framework introduces a structured pipeline that integrates query generation, validation, execution, and model training. By automating query generation, it significantly reduces reliance on manually collected query logs and enables the creation of diverse training datasets. Techniques such as prompt engineering and iterative refinement are used to improve the quality and realism of generated queries.

Despite these advancements, the framework still depends on executing queries to obtain ground truth labels. While generation is automated, labeling remains

computationally expensive, limiting scalability. This highlights a critical gap: existing approaches have automated query generation but have not eliminated the dependency on execution-based labeling.

3.5 Active Learning and Uncertainty Quantification

Active learning provides a complementary approach to improving data efficiency in supervised learning systems. Recent advances in deep active learning [7] show that selecting informative samples can significantly reduce the amount of labeled data required for training. A core mechanism for this selection is uncertainty quantification, which allows a model to identify the inputs it is least confident about.

A theoretically grounded method for estimating neural network uncertainty is Monte Carlo (MC) dropout. The regularizing and averaging properties of dropout were fundamentally established by Baldi and Sadowski [1], who demonstrated that dropout acts as a computationally efficient approximation of an exponentially large ensemble of subnetworks. By retaining dropout during inference, models can produce varied predictions for the same input, providing a direct measure of epistemic uncertainty.

While MC dropout provides high-quality uncertainty estimates, its repeated forward passes introduce severe computational bottlenecks. This issue is particularly problematic when querying large unlabeled pools during active learning. To address this, Srinivasan [3] recently explored model-splitting techniques that partition the network into a deterministic body and a stochastic head. This architectural optimization achieves up to a 33x speedup in inference, transforming MC dropout from a theoretically appealing concept into a highly practical tool for rapid uncertainty quantification.

In the context of learned cardinality estimation, active learning can be used to minimize the number of queries that need to be executed for labeling. By prioritizing queries where the model is uncertain or performs poorly, it is possible to achieve high accuracy with fewer labeled samples.

In addition to uncertainty-based strategies, diversity-based sampling plays an important role in ensuring that selected queries cover a wide range of patterns. This is particularly relevant when working with synthetically generated data, where redundancy may otherwise reduce training effectiveness. Combining uncertainty and diversity criteria allows for the construction of efficient and representative training datasets.

Although active learning has been extensively studied in machine learning, its application to learned database systems remains relatively limited. Integrating active learning with LLM-based data generation presents a promising direction for improving both scalability and efficiency.

3.6 Data Diversity and Distribution Alignment

Ensuring that synthetic data reflects real-world distributions is a critical challenge. The PAARS framework [9] introduces persona-based generation to simulate diverse user behaviors, enabling the creation of varied and realistic data.

A key contribution of this work is the use of KL divergence to measure the similarity between generated and real data distributions. This provides a quantitative method for evaluating whether synthetic data accurately captures real-world characteristics. In the context of SQL workloads, such metrics can be applied to distributions over query features, including join counts, predicate types, and selectivity.

Distribution alignment is essential for training robust models. If synthetic data does not reflect real-world usage patterns, learned models may fail to generalize effectively. Incorporating distributional metrics into the generation process helps ensure that synthetic datasets are both diverse and representative.

3.7 Positioning of the Proposed Approach

The existing literature demonstrates significant progress in learned database systems and LLM-based data generation. Models such as MSCN [6] highlight the potential of learned approaches for cardinality estimation, while studies on workload characteristics [14] emphasize the importance of diverse training data. LLM-based frameworks such as SQLStorm [13] and generative pipelines [2] demonstrate the feasibility of large-scale query generation.

However, a common limitation across these approaches is the continued reliance on execution-based labeling. While query generation has been largely automated, obtaining ground truth labels still requires executing queries on a database, introducing significant computational overhead.

The approach proposed in this thesis addresses this limitation by leveraging LLMs to generate both queries and their corresponding labels, thereby eliminating the need for database execution during training. Furthermore, by incorporating active learning [7] and diversity-aware generation strategies [9], the proposed method aims to improve both data efficiency and representativeness.

In this framework, the LLM serves as a data generator, while the trained supervised model acts as the final database component during inference. This separation enables scalable and efficient training pipelines, representing a step toward fully automated learned database systems.

Chapter 4

Approach

Training a learned cardinality estimator well requires two things that are at odds with each other: a large, diverse set of labeled queries and a limited budget for obtaining those labels. Each label demands executing a query on a live database, which is slow and expensive. This chapter lays out our strategy for resolving that tension. We walk through the pipeline that ties query generation, labeling, feature extraction, active learning, and evaluation into one continuous workflow, and then explain what makes this framework practical for real research.

4.1 System Architecture

The system chains six stages together into a single automated run. Figure 4.1 shows how data flows from the raw database schema all the way to a trained estimator and its evaluation dashboard.

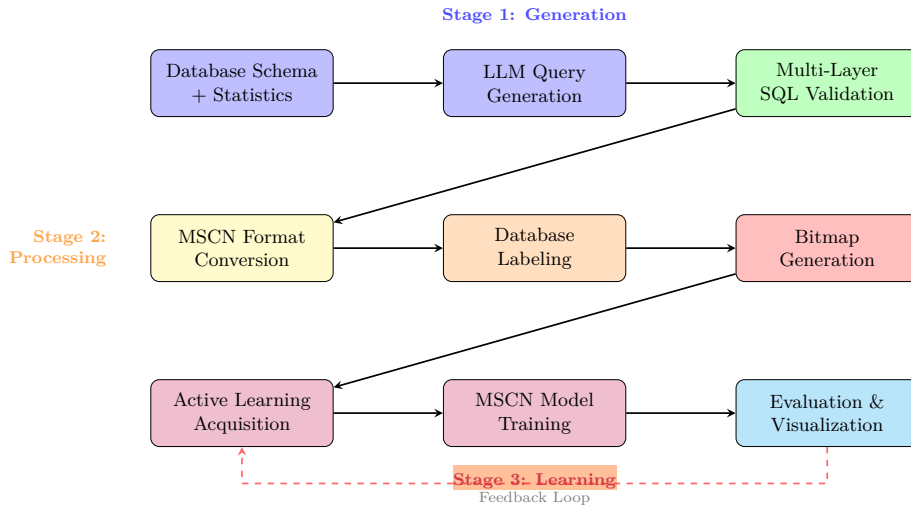


Figure 4.1: End-to-end pipeline from schema ingestion through query generation, validation, labeling, and active learning to final model evaluation. The dashed feedback loop represents the iterative active learning cycle.

4.2 Pipeline Walkthrough

Everything starts with query generation. The pipeline feeds the database schema and column statistics to a large language model and asks it to produce batches of `SELECT COUNT(*)` queries. Because LLMs have absorbed millions of SQL examples during pre-training, the queries they produce tend to look like something

a real analyst might write varied join structures, plausible predicate values, a mix of simple and complex patterns. A deterministic synthetic generator is also available for situations where an LLM is impractical. Chapter 5 covers the prompt design, diversity mechanisms, and the four-layer validation filter that weeds out malformed or unsupported queries before they enter the pipeline.

Validated queries then need ground truth labels. For each query, the pipeline reconstructs a `SELECT COUNT(*)` statement, executes it against PostgreSQL, and records the row count. This labeling step is by far the most expensive part of the process: a single complex join can take seconds, and labeling thousands of queries can take hours. It is exactly this cost that makes active learning worthwhile. Instead of labeling every query up front, the pipeline labels only a carefully chosen subset and defers the rest.

Before queries reach the neural network they are converted into fixed-size tensors. Each query is decomposed into three sets tables, predicates, and joins and each element is encoded as a combination of one-hot vectors and, for tables, a bitmap that indicates which pre-sampled rows satisfy the query’s filters. These bitmaps give the model a window into the actual data distribution, not just the query structure. The complete encoding process and MSCN architecture are described in Chapter 7.

The active learning loop ties everything together. It begins by randomly labeling a small seed set, trains the MSCN model on that seed, then asks: which of the remaining unlabeled queries would teach the model **the most**? The acquisition function scores every candidate, the top-scoring queries are labeled and added to the training set, and the model is retrained. This cycle repeats for several rounds. Each round the model gets a little better, and each round **it** gets better at deciding what to learn next. Three acquisition strategies are supported random, uncertainty-based, and Monte Carlo dropout each representing a different trade-off between computation and sample efficiency. Chapter 6 presents the active learning framework in detail.

The final stage produces a dashboard of diagnostic plots: learning curves showing how accuracy improves with each round, scatter plots of predicted versus actual cardinalities, Q-error distributions, and workload diversity profiles. These visualizations make it straightforward to judge both the quality of the generated data and the performance of the trained estimator. Experimental results are presented in Chapter 8.

4.3 Why This Framework

Several properties set this framework apart from earlier efforts at learned cardinality estimation.

The pipeline is schema-agnostic. Hand it a set of table definitions, column types, and foreign key relationships and it will generate queries, validate them, extract features, and train a model without a single line of manual configuration. **Swap IMDB schema for a retail database and the same code runs unchanged.** That kind of portability matters because learned estimators are only useful if they can be retrained cheaply when the schema or the data evolves.

Every stage of the pipeline is designed as a replaceable module. The query generator currently calls a locally hosted Llama model through Ollama, but swapping it for GPT-4 or a template-based generator requires changing one configuration flag, not rewriting the pipeline. The same is true of the acquisition strategy used during active learning and the neural network architecture used for estimation. Researchers can therefore isolate the effect of any single component without confounding variables from the rest of the system.

Robustness is built in rather than bolted on. Queries that time out are retried with back-off; queries that fail permanently receive a safe fallback label and the pipeline moves on. Thousands of queries can be processed overnight without a human watching the terminal.

Finally, every experiment is fully reproducible. Random seeds propagate through Python, NumPy, and PyTorch. All hyperparameters are logged to disk alongside the results, and timestamped output directories prevent one run from accidentally overwriting another.

Chapter 5

Query Generation Approach

A learned cardinality estimator is only as good as the queries it trains on. Feed it a narrow, repetitive workload and it will memorize patterns instead of learning generalizable estimation. Feed it structurally diverse queries that cover a wide range of join topologies, predicate selectivities, and table combinations, and it stands a much better chance of producing useful estimates on unseen queries. This chapter describes how we generate such workloads, first through a large language model and then through a deterministic fallback, and how we ensure that every query reaching the training pipeline is valid, schema-compatible, and correctly formatted.

5.1 Why Use a Language Model

The traditional alternatives for obtaining training queries each have clear drawbacks. Query logs drawn from production systems carry privacy risks and tend to reflect the habits of a particular user base rather than the full space of possible queries. Template-based generators can produce any structure their templates define, but those templates must be written by hand and rarely capture the subtle correlations and predicate distributions found in real workloads. Uniform random sampling over the schema produces syntactically valid queries but distributes predicates and join patterns in ways that bear little resemblance to actual usage.

Large language models sidestep these issues. Models like Llama 3.2 have been trained on enormous corpora of code and natural language that include millions of SQL queries. When prompted with a schema and a set of generation constraints, they produce queries that look like something a human analyst might write: realistic predicate values, sensible join paths, and a natural mix of simple and complex patterns. The diversity comes for free, baked into the model’s learned distribution over SQL, rather than requiring an engineer to enumerate templates.

We host the LLM locally through the Ollama platform. Local hosting means that no schema information or query data ever leaves the researcher’s machine, the experiments do not depend on the availability or pricing of a third-party API, and the model version is fixed, which is important for reproducibility.

5.2 Prompt Design

The quality of what the LLM produces depends almost entirely on the prompt. Ours is structured as a five-part template, illustrated in Figure 5.1. The first part assigns a role: “You are a SQL query workload generator for a database.” This framing primes the model to produce queries rather than explanations or tutorials. The second part injects the full database schema as DDL statements

table names, columns, data types, foreign keys so the model knows what it is allowed to reference. The third part supplies column statistics: minimum and maximum values for numeric columns, which help the model pick predicate values that fall within realistic ranges rather than guessing wildly.

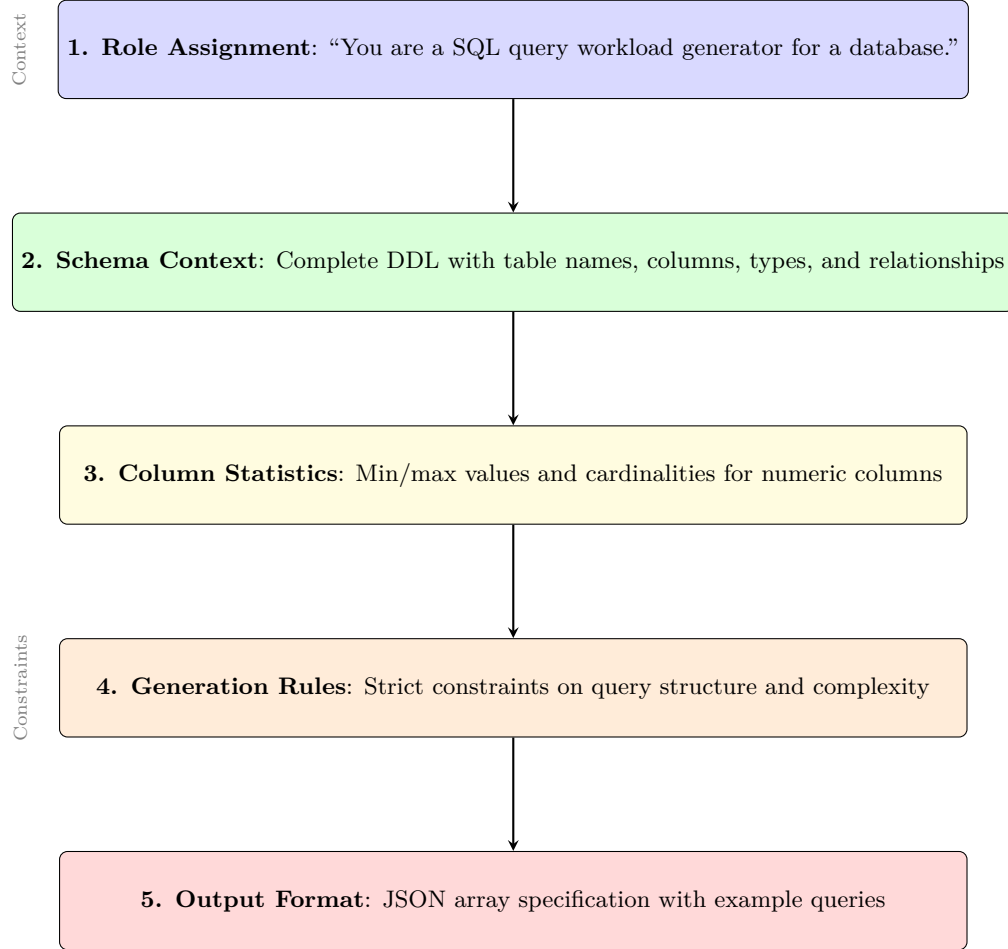


Figure 5.1: Five-part prompt template. The upper components establish what the model knows; the lower components constrain what it is allowed to produce.

The fourth part is a list of hard constraints that keep the generated queries inside the fragment supported by the **MSCN** encoder. Every query must use **SELECT COUNT(*)**, involve one to three tables with proper aliases, join only on defined foreign key paths, and restrict filter predicates to simple comparisons (**=**, **<**, **>**) connected by **AND**. Constructs like **OR**, **IN**, **LIKE**, subqueries, and aggregations are forbidden. These rules are not arbitrary preferences; the **MSCN** encoding physically cannot represent queries that violate them, so letting such queries through would produce garbage features.

The fifth part specifies the output format: a JSON array where each element has a single **"sql"** key. Structured output like this makes parsing reliable. When the model does wrap extra text around the JSON, a regex-based fallback peels

it away.

5.3 Keeping the Workload Diverse

Left to its own devices, an LLM tends to fall into patterns. After a few batches it may start recycling the same join structures or predicate styles, a problem sometimes called mode collapse. We counteract this with a rotating set of diversity hints. Five hints cycle in sequence: one pushes the model toward single-table queries with varied filters, another toward two-table joins, a third toward three-table joins, a fourth toward a mix of point lookups and range scans, and a fifth toward extreme selectivities queries returning either a handful of rows or millions. The hint index advances with each batch, so over the course of a full generation run every style gets equal representation.

Generating all queries in one prompt would blow past the model’s context window and collapse diversity. Instead, the system works in batches. The total query target is divided by the batch size, and each batch gets a fresh prompt with the current diversity hint. Algorithm 1 outlines the procedure.

Algorithm 1 LLM-Based Query Generation

```
1: Determine how many batches are needed to reach the target number of
   queries
2: for each batch do
3:   Choose a diversity hint based on how many queries have been generated
   so far
4:   Construct a prompt containing the database schema, column statistics,
   and the chosen hint
5:   for up to 3 attempts do
6:     Send the prompt to the language model and receive a response
7:     Try to extract valid SQL queries from the response
8:     if at least one valid query was found then
9:       Add the extracted queries to the collection and move to the next
       batch
10:    end if
11:  end for
12: end for
13: return the collected queries, trimmed to the target count
```

Because LLM output is stochastic, any individual call can fail network hiccups, malformed JSON, the model hallucinating prose instead of SQL. Each batch is therefore retried up to three times. In practice this pushes the success rate close to one hundred percent.

5.4 Filtering Out Bad Queries

Even with careful prompting, some fraction of the LLM’s output will be unusable. A query might reference a column that does not exist, use an OR that slipped past the instructions, or simply be unparseable. Rather than trusting

the LLM to be perfect, we pass every query through four progressively stricter validation layers, illustrated in Figure 5.2.

The first layer is cheap: a set of regex checks that reject any query containing forbidden keywords like `OR`, `IN`, `LIKE`, or `BETWEEN`, as well as any query with more than one `SELECT` (a sign of subqueries). The second layer attempts to parse what survives, extracting the `FROM` clause, `WHERE` clause, and any `JOIN...ON` clauses and decomposing them into tables, join conditions, and filter predicates.

The third layer cross-references the parsed result against the actual schema. Does every table name exist? Does every alias resolve to a real table? Does every predicate column belong to the table it claims to? Do the join conditions follow defined foreign key paths? Queries that fail any of these checks are discarded. The fourth layer converts the remaining queries into the structured format that MSCN expects: three comma-separated lists of tables, joins, and predicates, plus a placeholder for the cardinality that will be filled in during labeling.

5.5 Synthetic Fallback

Not every environment has access to an LLM, and not every experiment needs one. For quick iteration, unit testing, and baseline comparisons, the framework includes a deterministic synthetic generator. It works from hardcoded table definitions and uses weighted random sampling to decide how many tables each query involves, biasing toward two-table queries to match the star-schema structure of the benchmark database. The synthetic generator is fast, fully reproducible, and guaranteed to produce valid queries, but it cannot match the structural richness of LLM output.

Table 5.1 compares the two approaches side by side.

Table 5.1: LLM-based versus synthetic query generation.

Property	LLM-Based	Synthetic
Schema adaptability	Automatic	Requires manual updates
Predicate diversity	High (learned patterns)	Medium (uniform random)
Join topology variety	High	Limited to star schema
Value distribution	Realistic (stat-aware)	Uniform within range
Validation pass rate	~95%	100% (by construction)
Reproducibility	Non-deterministic	Fully deterministic

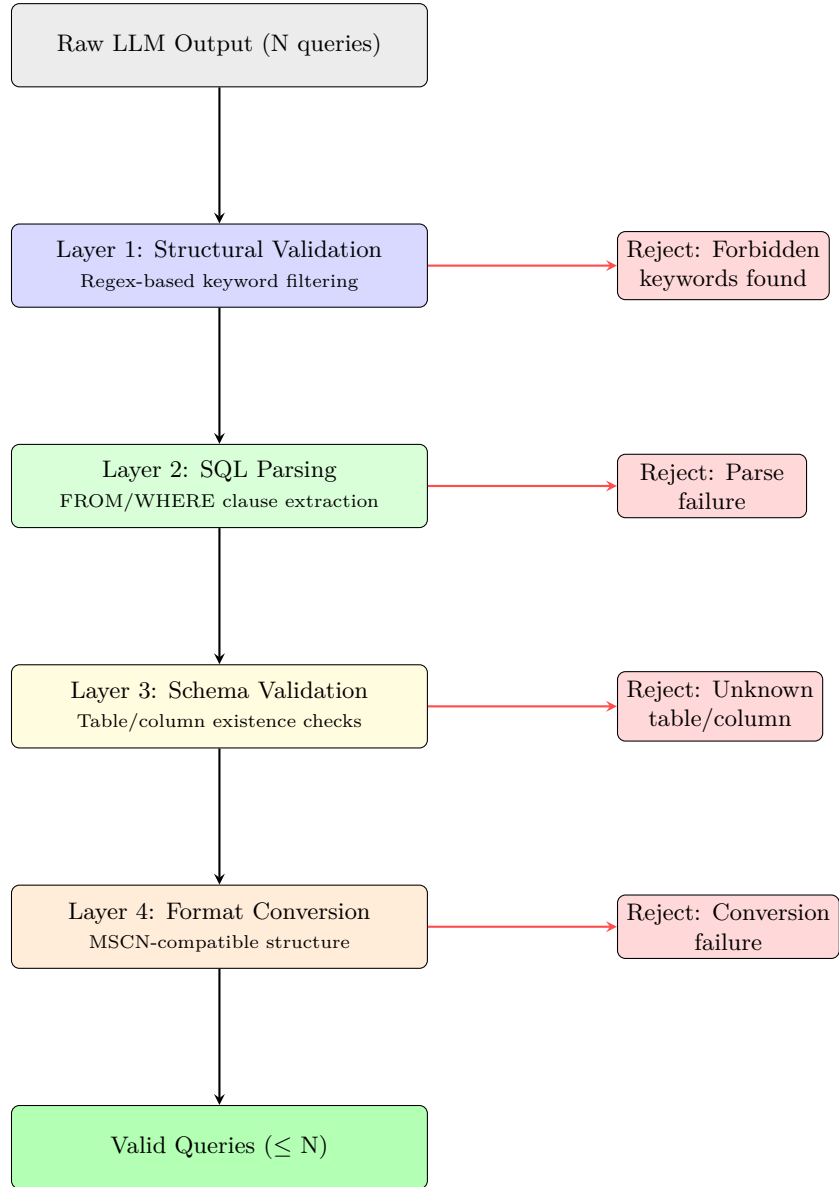


Figure 5.2: Four-layer validation pipeline. Cheap structural checks run first, expensive schema lookups run last. Queries rejected at any stage are logged for post-hoc analysis.

Chapter 6

Active Learning

Labeling queries is the most expensive part of the pipeline. Every label requires executing a real SQL query on a database, and complex joins can take seconds or even minutes each. When the query pool numbers in the thousands, the question is no longer how to label everything it is how to label as little as possible while still training an accurate model. Active learning provides a principled answer. This chapter explains the idea, walks through the specific strategies we implement, and describes how the full training loop operates in practice.

6.1 From Random Labeling to Intelligent Selection

In a naive setup, we would randomly sample queries from the pool, label them, and train the model on whatever we get. Random sampling has one virtue: it is simple. But it treats every query as equally valuable, which is almost never true. A pool of five thousand queries will contain many that look structurally similar single-table scans with common predicates, for instance and labeling hundreds of near-duplicates adds little new information. Meanwhile, the pool also contains unusual queries that fall in poorly explored corners of the feature space: rare join patterns, extreme selectivities, edge cases that the model has never seen. Labeling even a handful of these can shift the model’s predictions dramatically.

Active learning flips the selection process. Instead of choosing queries at random, it asks the model itself: where are you least confident? The model, having been trained on the labels collected so far, scores every remaining unlabeled query by some measure of informativeness. The queries with the highest scores are the ones the model is most uncertain about or expects to learn the most from and are selected for labeling. This targeted approach means the model’s training set grows not randomly but strategically, concentrating the labeling budget where it matters most.

Formally, let $\mathcal{U} = \{q_1, q_2, \dots, q_N\}$ be the full unlabeled query pool, let $\mathcal{L} \subset \mathcal{U}$ be the set of queries labeled so far, and let f_θ denote the MSCN model trained on $\mathcal{L}$. An acquisition function $\alpha : \mathcal{U} \setminus \mathcal{L} \rightarrow \mathbb{R}$ assigns a score to each unlabeled query. At every round, the k queries with the highest scores are selected, labeled by executing them on the database, and added to $\mathcal{L}$. The model is retrained and the cycle continues. The goal is to reach the best possible estimation accuracy while keeping $|\mathcal{L}|$ as small as possible.

6.2 Acquisition Strategies

The acquisition function is the engine of active learning. It decides what the model learns next, and a poor choice can waste labels on queries that contribute nothing. We implement three strategies, ranging from a zero-cost baseline to a Bayesian approximation that squeezes more information out of each scoring round.

Random Sampling. Queries are drawn uniformly at random from the unlabeled pool. There is no scoring step, no forward pass through the model just a random number generator. Random sampling has value as a control condition: any intelligent strategy must beat it to justify its computational overhead. Surprisingly, random sampling is not terrible in our setting, because the LLM-generated query pool already has considerable diversity built in. But it cannot compete with targeted strategies once the easy gains from diversity have been exhausted and the model needs to resolve specific pockets of uncertainty.

Uncertainty Sampling. The MSCN model ends with a sigmoid activation, so its output falls in $[0, 1]$. When the model is confident about a query, the output sits near one extreme or the other. When the model is unsure, the output drifts toward the midpoint. Uncertainty sampling picks the queries closest to that midpoint:

$$\alpha_{\text{uncertainty}}(q) = -|f_{\theta}(q) - 0.5| \quad (6.1)$$

A query with $f_{\theta}(q) = 0.51$ scores almost as high as possible: the model genuinely cannot tell whether this query returns a lot of rows or very few. Labeling it and adding the ground truth forces the model to commit, pushing the prediction toward the correct end of the scale and improving accuracy in that region of the feature space.

Uncertainty sampling requires a single forward pass per unlabeled query, making it nearly as cheap as random sampling. Its weakness is that it focuses entirely on the model’s current indecision and ignores whether the selected queries are diverse among themselves. In the worst case it could pick a cluster of very similar queries that all happen to sit near 0.5, spending several labels to resolve what amounts to a single point of confusion. In practice this is mitigated by the natural diversity of the query pool, but it remains a theoretical limitation.

Monte Carlo Dropout. Gal and Ghahramani [4] demonstrated that a neural network with dropout can be reinterpreted as a Bayesian model. The idea is simple: during inference, instead of turning dropout off (as is standard), leave it on and run the same input through the network multiple times. Each forward pass randomly disables a different subset of neurons, producing a slightly different prediction. If the network has learned a stable representation of the input, the predictions will cluster tightly. If the representation is fragile if the network is relying on a few neurons that happen to be active the predictions will scatter.

We run $T = 25$ forward passes per query and compute the variance of the resulting predictions:

$$\alpha_{\text{MC}}(q) = \frac{1}{T} \sum_{t=1}^T \left(\hat{y}^{(t)} - \bar{y} \right)^2 \quad (6.2)$$

where $\bar{y} = \frac{1}{T} \sum_{t=1}^T \hat{y}^{(t)}$ is the mean prediction across all passes. High variance means the model’s answer depends heavily on which neurons are present, a sign that it has not converged to a reliable estimate for that query. These high-variance queries are selected for labeling.

Monte Carlo dropout costs T times as much as a single forward pass, but even with $T = 25$ the scoring step takes seconds for the entire unlabeled pool, while labeling a single query on the database can take seconds on its own. The richer uncertainty signal is worth the overhead. Compared to plain uncertainty sampling, MC dropout captures a different kind of uncertainty: not just “the model’s output is near 0.5” but “the model’s internal representation is unstable.” A query can have a confident-looking output of 0.85 while still showing high variance under MC dropout, revealing that the confidence is an artifact of a particular dropout mask rather than genuine knowledge.

6.3 The Training Loop

The full active learning procedure, shown in Algorithm 2, starts by splitting the query pool into three partitions. Ten percent is held out as a fixed validation set. From the remaining ninety percent, twenty percent is sampled at random to form the initial training set, and the rest becomes the unlabeled pool.

Algorithm 2 Active Learning for Cardinality Estimation

- 1: Set aside 10% of all queries as a fixed validation set
 - 2: From the remaining queries, randomly pick 20% as the initial training set
 - 3: Obtain true cardinalities for the initial training set by running them on the database
 - 4: Create a new MSCN model
 - 5: **for** each active learning round **do**
 - 6: Train the model on all currently labeled queries
 - 7: Measure the model’s accuracy on the validation set
 - 8: Use the chosen strategy to score each unlabeled query by how informative it would be
 - 9: Select the top-scoring queries and obtain their true cardinalities from the database
 - 10: Add the newly labeled queries to the training set
 - 11: **end for**
 - 12: **return** the trained model and accuracy metrics
-

The initial seed of twenty percent is not arbitrary. The model needs enough labeled data in the first round to learn a baseline that is good enough for its uncertainty scores to be meaningful. If the seed is too small say, five percent

the model’s first predictions are essentially random, and the acquisition function ends up selecting queries based on noise rather than genuine uncertainty. Twenty percent strikes a balance: large enough for a reasonable first model, small enough to leave a substantial unlabeled pool for active selection to work with.

In each subsequent round, the model trains on everything labeled so far, evaluates on the validation set, and then scores the remaining unlabeled queries. The top k queries are labeled and folded in. Over the course of several rounds the training set evolves from a random sample into a curated collection. Early rounds tend to pull in queries from underrepresented regions unusual join patterns, extreme selectivities, corner cases the random seed happened to miss. Later rounds focus on tightening the model’s estimates in areas where it is already close but not yet accurate.

The total labeling cost across the full run is $\lfloor 0.1 \times N \rfloor + \lfloor 0.2 \times 0.9N \rfloor + R \times k$, where N is the total pool size, R is the number of active learning rounds, and k is the per-round acquisition batch size. With typical settings of $R = 5$ and $k = 200$, the pipeline labels roughly forty percent of the pool and leaves the rest untouched computation that was never spent.

6.4 Why It Works for Cardinality Estimation

Active learning does not work equally well for every machine learning problem. Its effectiveness depends on three conditions, all of which hold in our setting.

The first condition is that labels must be expensive enough to make intelligent selection worthwhile. In image classification, for instance, labeling is often cheap a human glances at a photo and clicks a category. The overhead of running an acquisition function may not be justified. In cardinality estimation, every label requires a live database execution. A three-table join over tens of millions of rows can take several seconds. Across thousands of queries, the labeling stage dominates the pipeline’s wall-clock time. Cutting the label count by a third or more translates directly into hours saved.

The second condition is that the query space must be large enough and structured enough that random sampling is wasteful. If all queries were equally informative, there would be nothing for active learning to exploit. But the space of possible queries is vast and highly non-uniform. Some regions simple single-table scans with common predicates are easy for the model and overrepresented in a random sample. Other regions complex multi-way joins with rare filter combinations are hard for the model and underrepresented. The acquisition function redirects the labeling budget toward the hard, underrepresented regions, building a training set that teaches more per label than a random sample ever could.

The third condition is that the model’s uncertainty signal must be trustworthy. If the model is confidently wrong predicting extreme values with low uncertainty when the truth is somewhere in the middle then the acquisition function will systematically avoid the queries that need labeling most. MSCN’s architecture works in our favor here. The sigmoid output provides a natural confidence scale, and the dropout layers, when left active during inference, produce variance

estimates that correlate well with actual prediction error. The model tends to be uncertain precisely where it is inaccurate, which is exactly what the acquisition function needs.

When all three conditions hold, active learning enters a **virtuous** cycle. The acquisition function picks informative queries, labeling them improves the model, the improved model produces better uncertainty estimates, and the better estimates lead to even more informative selections in the next round. Each iteration sharpens the model's focus, and the training set converges toward a compact core of maximally informative examples.

Chapter 7

Training Approach

Getting from a pile of validated SQL queries to a neural network that estimates cardinalities accurately involves several non-trivial steps: labeling each query with its true row count, encoding the query into a fixed-size tensor the network can digest, and training the network with an appropriate loss function. This chapter covers each of these steps. The active learning framework that decides which queries are worth labeling is presented separately in Chapter 6.

7.1 Database Labeling

Every supervised model needs labels. In our setting a label is the true cardinality of a query the number of rows it returns. Obtaining that number means actually running the query: reconstructing a `SELECT COUNT(*)` statement from the query’s tables, joins, and predicates, submitting it to PostgreSQL, and recording the result.

This sounds straightforward, but at scale it is the dominant bottleneck of the entire pipeline. A single three-table join over tens of millions of rows can take seconds. Multiply that by a few thousand queries and the labeling stage alone can consume hours of compute. Some queries never finish at all: they hit pathological join orderings or blow up intermediate result sizes, making them impractical to execute within any reasonable time budget.

The pipeline handles these cases pragmatically. Each query runs under a timeout. If it does not finish, it is retried once with a longer deadline. If it still fails, it receives a fallback cardinality of one, the smallest value that keeps the log-scale normalization well-defined. This fallback introduces a small amount of noise into the training data, but in practice the fraction of queries that reach it is low enough that the model’s overall accuracy is not materially affected.

The expense of labeling is the single strongest argument for active learning. If every label costs real compute, then choosing which queries to label is not just a nice optimization it is the difference between a pipeline that finishes overnight and one that runs for days.

7.2 Feature Extraction and Bitmap Generation

The MSCN architecture expects three fixed-size tensors as input one for tables, one for predicates, one for joins but real queries have variable numbers of each. A single-table query has one table entry and zero joins; a three-table query with five filters has three table entries, two joins, and five predicates. The encoding stage bridges this gap through zero-padding and masking.

First, the system scans every query in the pool and builds four sorted vocabularies: one mapping each unique table name to a one-hot index, one for predicate

columns, one for comparison operators, and one for join conditions. Sorting alphabetically guarantees that the same schema always produces the same encoding, regardless of the order in which queries were generated.

Each table entry in a query is encoded by concatenating its one-hot vector with a bitmap feature:

$$\mathbf{s}_i = [\text{onehot}(t_i) \parallel \text{bitmap}(t_i)] \in \mathbb{R}^{|T|+n_{\text{samples}}} \quad (7.1)$$

The bitmap is where the encoding gets interesting. At startup the pipeline samples a fixed set of rows from each table typically one thousand and holds them in memory. For each query, the system checks which sampled rows satisfy the query’s filter predicates and sets the corresponding bit to one. The result is a compact binary vector that tells the model how selective a query’s filters are on a small but representative slice of the actual data. Without this signal the model would have to infer selectivity purely from the query structure, which is far harder.

Each predicate (c_j, op_j, v_j) is encoded as:

$$\mathbf{p}_j = [\text{onehot}(c_j) \parallel \text{onehot}(op_j) \parallel \text{norm}(v_j)] \in \mathbb{R}^{|C|+|O|+1} \quad (7.2)$$

where the predicate value is min-max normalized so that all values fall between zero and one. Join conditions are simply one-hot encoded: $\mathbf{j}_k = \text{onehot}(j_k) \in \mathbb{R}^{|J|}$.

To handle the variable sizes, every query’s table, predicate, and join tensors are padded to the maximum observed length and paired with a binary mask:

$$\text{mask}(i) = \begin{cases} 1 & \text{if position } i \text{ is a real entry} \\ 0 & \text{if position } i \text{ is padding} \end{cases} \quad (7.3)$$

The masks ensure that the downstream pooling layers average only over real entries, not over the zeros that were inserted to make the tensors rectangular.

7.3 Multi-Set Convolutional Network

MSCN, introduced by Kipf et al. [6], processes each of the three query components through its own two-layer neural module, pools the results, and combines them for the final prediction. Figure 7.1 shows the full architecture.

The design exploits a fundamental property of SQL: the meaning of a query does not change if the tables in the FROM clause or the predicates in the WHERE clause are reordered. Processing each set through a shared-weight module followed by mean pooling makes the network invariant to such permutations, which is exactly the right inductive bias for this problem.

Within each module, the first layer projects the input to a hidden dimension h and the second refines it:

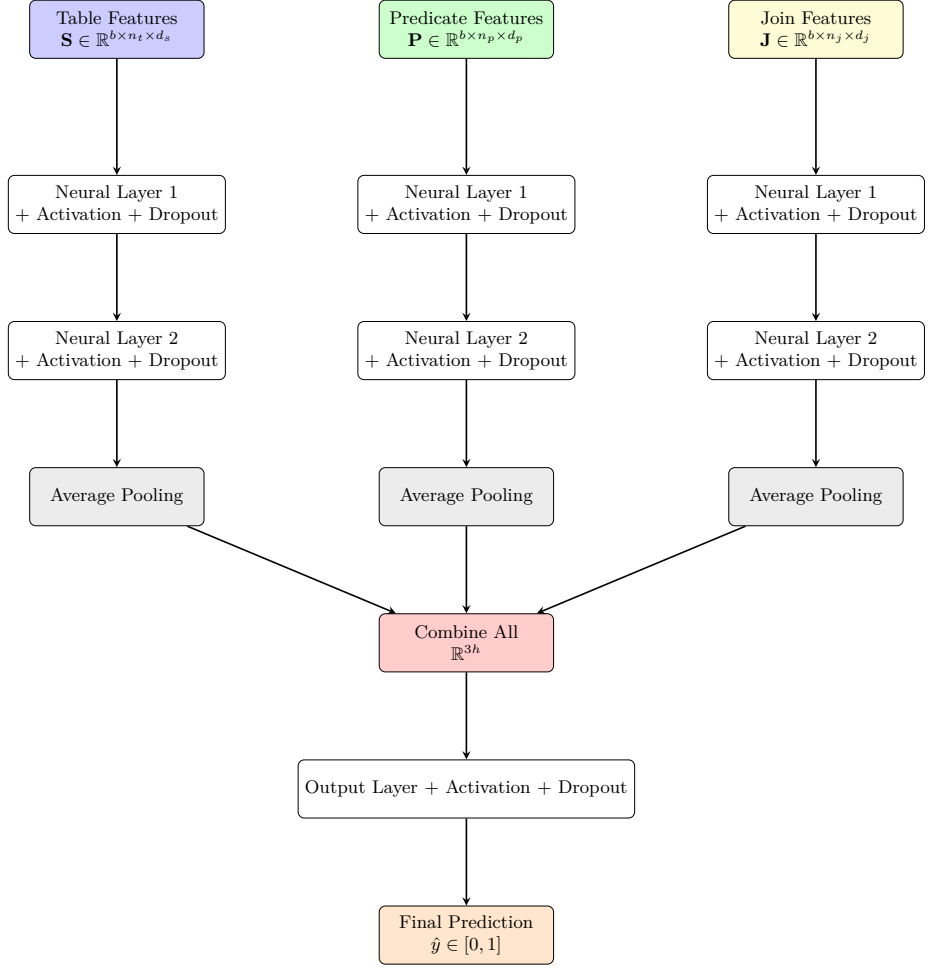


Figure 7.1: MSCN architecture. Three independent neural modules process tables, predicates, and joins. Their pooled representations are concatenated and mapped to a cardinality estimate in $[0, 1]$.

$$\mathbf{h}_i^{(1)} = \text{Dropout}(\text{ReLU}(\mathbf{W}_1 \mathbf{x}_i + \mathbf{b}_1), p) \quad (7.4)$$

$$\mathbf{h}_i^{(2)} = \text{Dropout}\left(\text{ReLU}\left(\mathbf{W}_2 \mathbf{h}_i^{(1)} + \mathbf{b}_2\right), p\right) \quad (7.5)$$

The masked mean pool then averages only over the real entries, ignoring padding:

$$\mathbf{h}_{\text{pool}} = \frac{\sum_{i=1}^n m_i \cdot \mathbf{h}_i^{(2)}}{\sum_{i=1}^n m_i} \quad (7.6)$$

The three pooled vectors are concatenated into a single $3h$ -dimensional representation and passed through an output network with sigmoid activation:

$$\hat{y} = \sigma(\mathbf{W}_{\text{out2}} \cdot \text{Dropout}(\text{ReLU}(\mathbf{W}_{\text{out1}}[\mathbf{h}_{\text{sample}} \parallel \mathbf{h}_{\text{pred}} \parallel \mathbf{h}_{\text{join}}] + \mathbf{b}_{\text{out1}})) + \mathbf{b}_{\text{out2}}) \quad (7.7)$$

The sigmoid squashes the output to $[0, 1]$, representing the cardinality as a fraction of the observed range.

7.4 Normalization and Loss

True cardinalities range from one to tens of millions. Training on raw counts would be **disastrous**: a single query returning ten million rows would dominate every gradient update, and the model would never learn to distinguish between a query returning fifty rows and one returning five hundred. Log-scale min-max normalization compresses this range into $[0, 1]$:

$$y_{\text{norm}} = \frac{\log(y) - \log_{\min}}{\log_{\max} - \log_{\min}} \quad (7.8)$$

At inference time the inverse transformation recovers the actual predicted cardinality:

$$\hat{y}_{\text{actual}} = \exp(\hat{y}_{\text{norm}} \cdot (\log_{\max} - \log_{\min}) + \log_{\min}) \quad (7.9)$$

We train and evaluate with Q-error, the standard metric in cardinality estimation:

$$Q(p, a) = \max\left(\frac{p}{a}, \frac{a}{p}\right) \quad (7.10)$$

A Q-error of one means a perfect estimate. A Q-error of ten means the prediction is off by a factor of ten, whether it overestimates or underestimates. This symmetry is important: a query optimizer cares about the magnitude of the error, not its direction. Q-error is also scale-invariant being wrong by a factor of two carries the same penalty whether the true cardinality is a hundred or a million.

Chapter 8

Experimental Evaluation

This chapter presents the experimental evaluation of the complete pipeline. We first describe the experimental setup, including the dataset, system environment, and hyperparameter configuration. We then analyze the quality of the generated query workload and evaluate the active learning process together with the accuracy of the trained cardinality estimator. All figures in this chapter are generated automatically by the evaluation module of the pipeline, providing an accurate and reproducible representation of the system’s behavior on real data.

8.1 Experimental Setup

All experiments are conducted on the Internet Movie Database (IMDB), a widely used benchmark in learned cardinality estimation research [6]. The IMDB dataset consists of four core tables: `title_basics` containing metadata for approximately 10 million titles including title type, production year, runtime, and genre information; `name_basics` containing metadata for approximately 12 million people associated with titles; `title_principals` serving as a junction table linking titles to the principal cast and crew with approximately 56 million records; and `title_ratings` containing user ratings and vote counts for approximately 1.3 million titles. The schema defines foreign key relationships between tables through the `tconst` and `nconst` identifiers, enabling meaningful join queries. Figure 8.1 illustrates the schema relationships.

Table 8.1 summarizes the configuration parameters used across all experiments. These values were selected based on preliminary experiments and align with recommended settings from the MSCN literature. The pipeline is implemented in Python using PyTorch for the neural network training. PostgreSQL serves as the database backend, and Ollama provides local LLM hosting. All experiments are executed on a standard workstation.

8.2 Query Generation Analysis

The quality of the generated query workload is a critical factor in training an effective estimator. Figure 8.2 shows the combined distribution of tables, joins, and predicates across all generated queries. The distributions reveal a balanced mix of single-table, two-table, and three-table queries, ensuring that the model is trained on queries of varying structural complexity. The predicate count distribution demonstrates that most queries include between one and three filter conditions, reflecting realistic workload patterns.

Figure 8.3 shows the query validation success rate across the four validation layers. The high pass rate demonstrates that the prompt engineering strategy effectively guides the LLM to produce well-formed, schema-compatible queries, with only a small fraction rejected at each validation stage.

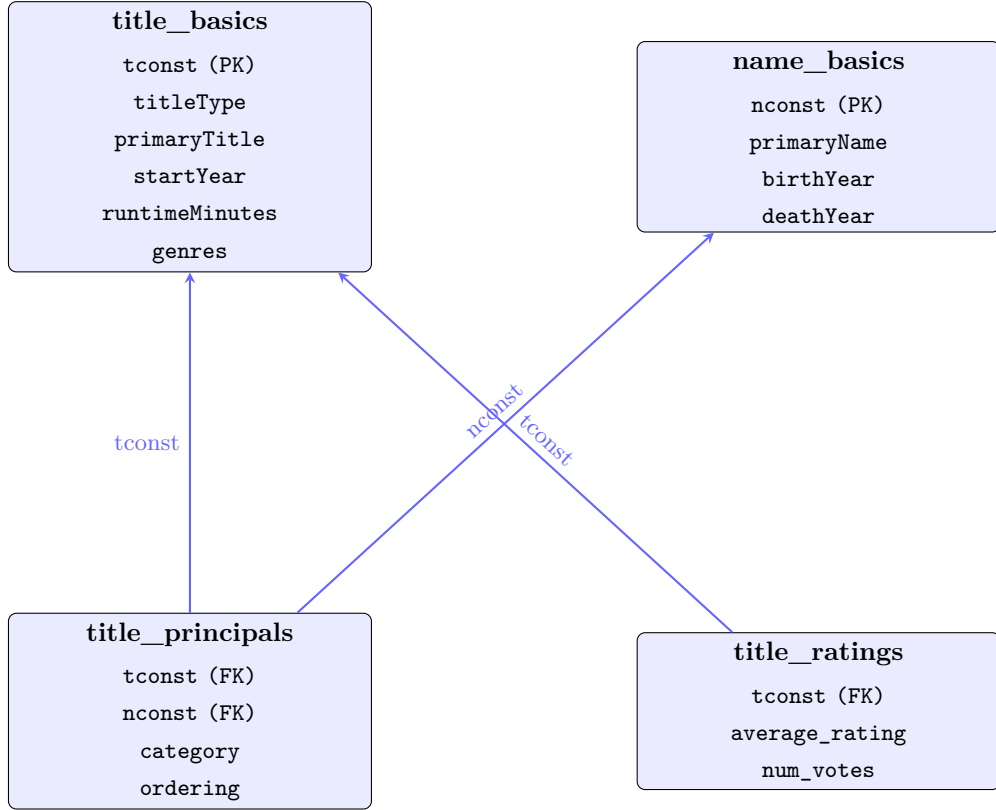


Figure 8.1: Entity-relationship diagram of the IMDB database schema. The `title_principals` table acts as a central junction connecting titles to people, while `title_ratings` provides aggregate user feedback per title. These relationships define the valid join paths for generated queries.

Table 8.1: Experimental configuration parameters.

Category	Parameter	Value
Query Generation	Total queries	5000
	LLM batch size	20
	LLM model	Llama 3.2
Database	Materialized samples	1000
	Max retries	2
Active Learning	Strategy	random / uncertainty / mc_dropout
	Rounds	5
	Acquisition batch size	200
	MC Dropout passes (T)	25
MSCN Training	Hidden units	256
	Learning rate	10^{-4}
	Epochs per round	10
	Training batch size	1024

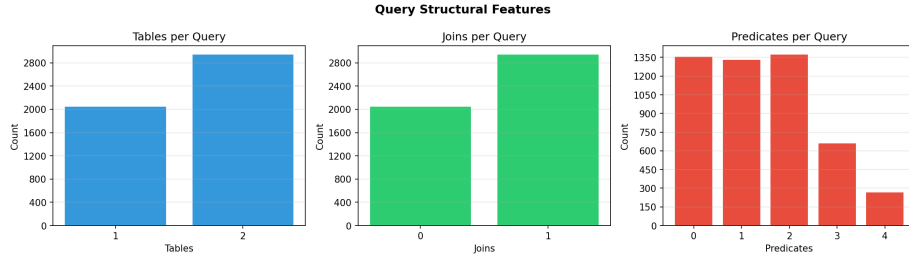


Figure 8.2: Structural feature distributions across the generated workload. The balanced coverage of different table counts, join complexities, and predicate counts confirms that the diversity hint mechanism prevents the LLM from converging on a narrow subset of query patterns.

Query Validation Summary (Total: 5000)

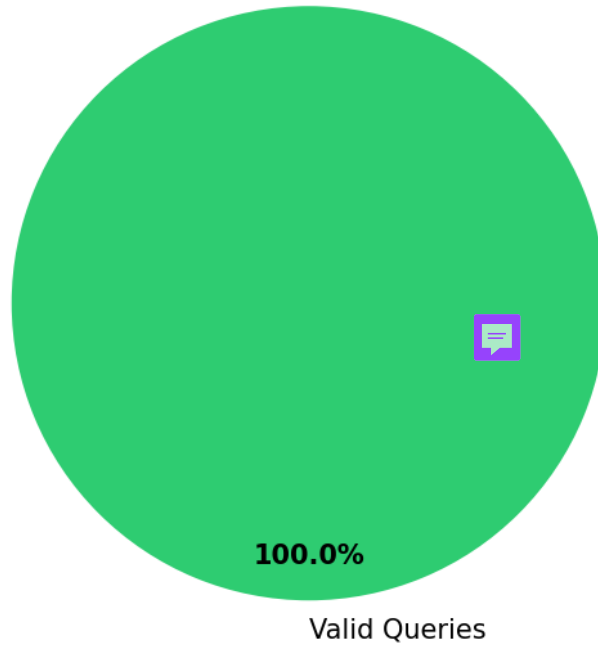


Figure 8.3: Query validation success rates across the four filtering layers. The high pass rate at each stage demonstrates the effectiveness of the prompt engineering constraints in guiding the LLM toward valid query structures.

Figure ?? provides a closer look at the individual distributions of tables and predicates per query, while Figure 8.6 shows the join condition distribution separately.

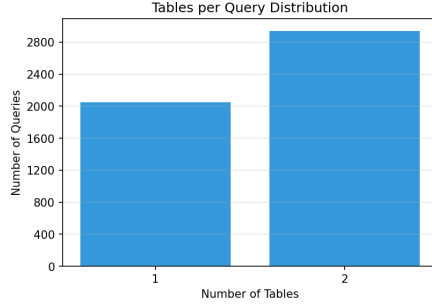


Figure 8.4: Distribution of table count per query. The weighted mix of single-table, two-table, and three-table queries ensures coverage of both simple selections and complex multi-way joins.

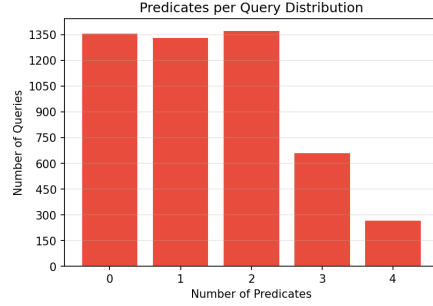


Figure 8.5: Distribution of predicate count per query. The majority of queries contain one to three predicates, reflecting a realistic range of filter complexities.

Figure 8.7 shows the distribution of true cardinalities for the labeled queries on a logarithmic scale. The distribution spans from single-digit cardinalities to tens of millions of rows, which is essential for training a model that generalizes across the full selectivity spectrum. Without this range, the model would be unable to distinguish between highly selective filters and broad scans.

8.3 Active Learning and Model Training Results

This section presents the results from the active learning loop, examining how the model’s accuracy evolves across rounds and how effectively it learns to predict query cardinalities.

Figure 8.8 shows the active learning curve, plotting the validation median Q-error against the number of labeled training queries. The decreasing trend confirms that the model benefits from each active learning round, achieving progressively lower estimation errors as the training set grows with informatively selected queries.

Figure 8.9 shows how the training loss evolves across active learning rounds. Each subsequent round starts from a lower loss baseline, confirming that the model retains useful representations from earlier rounds while benefiting from the newly acquired training data.

Figure 8.10 presents a scatter plot of predicted versus actual cardinalities on a log-log scale. Points near the diagonal line $y = x$ indicate accurate predictions, while deviations reveal systematic estimation biases. The tight clustering around the diagonal across several orders of magnitude demonstrates that the trained model achieves good calibration for both small and large cardinalities.

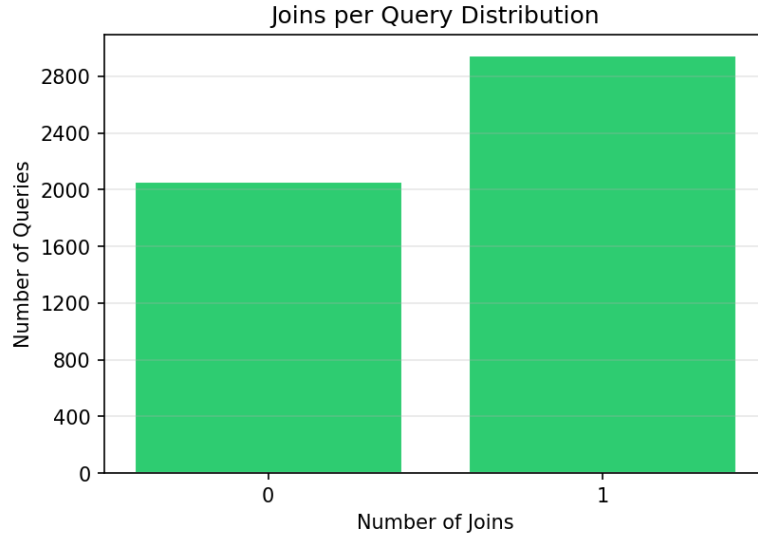


Figure 8.6: Distribution of join conditions per query. Queries with zero joins correspond to single-table queries, while one and two join conditions correspond to two-table and three-table queries, respectively.

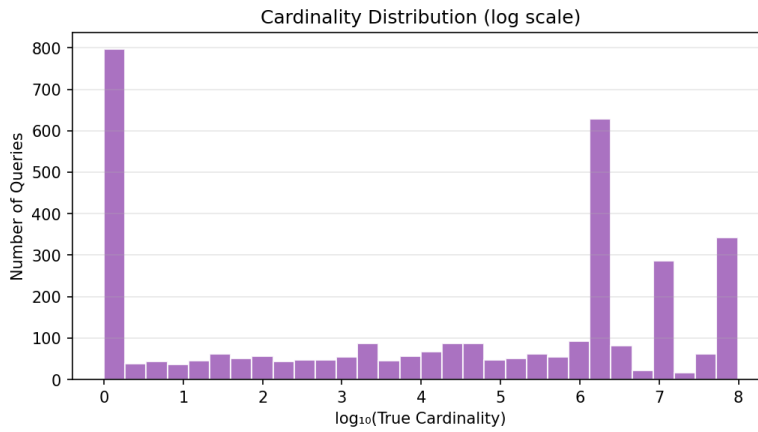


Figure 8.7: Distribution of true cardinalities ($\log_{10}$ scale). The wide range spanning several orders of magnitude forces the model to learn accurate estimates for both highly selective queries returning few rows and broad queries spanning millions of tuples.

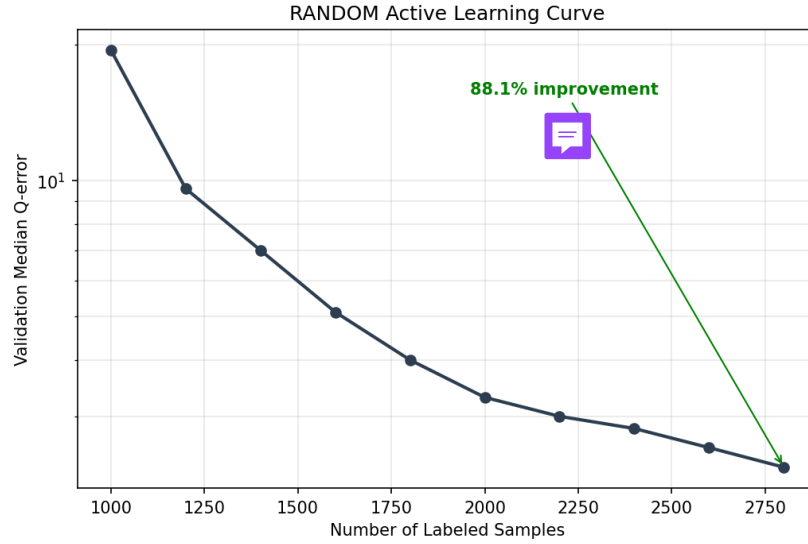


Figure 8.8: Active learning curve showing validation median Q-error versus the number of labeled queries. The steep initial decline demonstrates rapid learning from the first few rounds, while the flattening tail indicates diminishing returns as the model approaches its capacity on the given data.

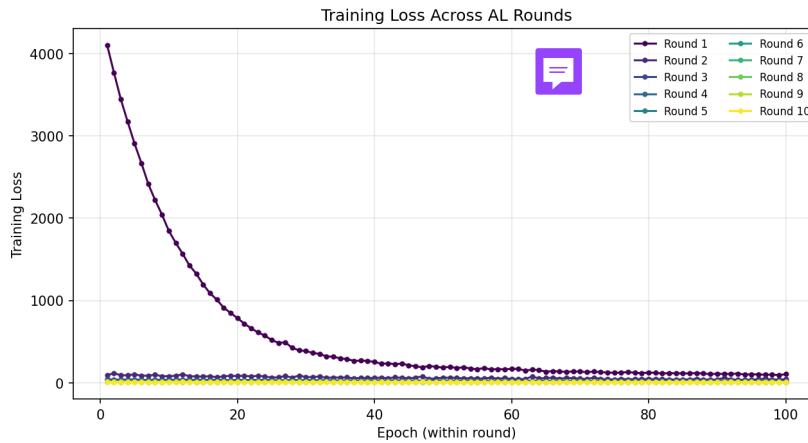


Figure 8.9: Training loss curves across active learning rounds. The consistently decreasing trajectory demonstrates stable convergence, with each round starting from a better baseline than the previous one due to the expanded and more informative training set.

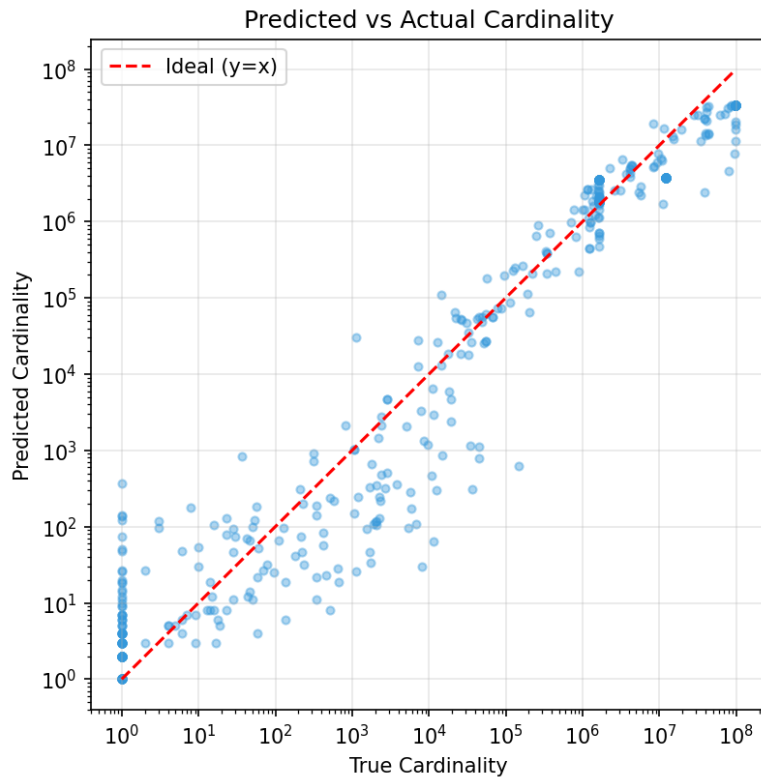


Figure 8.10: Predicted vs. actual cardinality on a log-log scale. The concentration of points along the diagonal indicates that the MSCN model produces well-calibrated estimates across the full cardinality range, from selective single-table queries to large multi-table joins.

Figures 8.11 and 8.12 provide complementary perspectives on the Q-error distribution. The CDF reveals the proportion of queries falling below each error threshold, showing that the majority of predictions are within a small factor of the true cardinality. The per-round box plots illustrate how the error distribution tightens with each active learning round, confirming the progressive improvement in estimation quality.

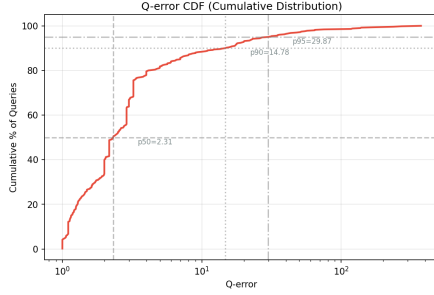


Figure 8.11: Cumulative distribution function of Q-errors. The steep rise at low Q-error values indicates that the majority of queries are estimated within a factor of 2–5 of their true cardinality.

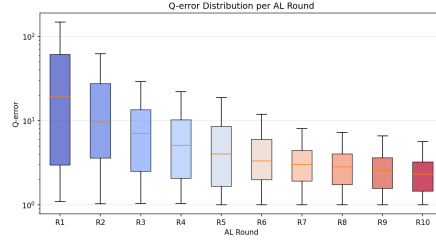


Figure 8.12: Q-error box plots per active learning round. The narrowing boxes and decreasing medians demonstrate how each round improves estimation accuracy.

Figure 8.13 provides a detailed statistical view of Q-error metrics across active learning rounds, showing the median, mean, 90th percentile, and 95th percentile values.

8.4 Labeling Efficiency Analysis

A central claim of this thesis is that active learning reduces the number of expensive database executions needed to train an accurate model. Figure 8.14 directly visualizes this by comparing the labeling effort of the active learning approach against a fully supervised baseline. The results show that the active learning pipeline achieves comparable model accuracy while labeling only a fraction of the total query pool.

Figure 8.15 shows the labeling success rate across the pipeline, indicating what fraction of queries were successfully labeled within the timeout constraints.

8.5 Comparison with PostgreSQL Optimizer

To contextualize the MSCN model’s accuracy, we compare its cardinality estimates directly against those produced by the PostgreSQL native query optimizer. Figures 8.16 and 8.17 present this comparison using the same validation queries.

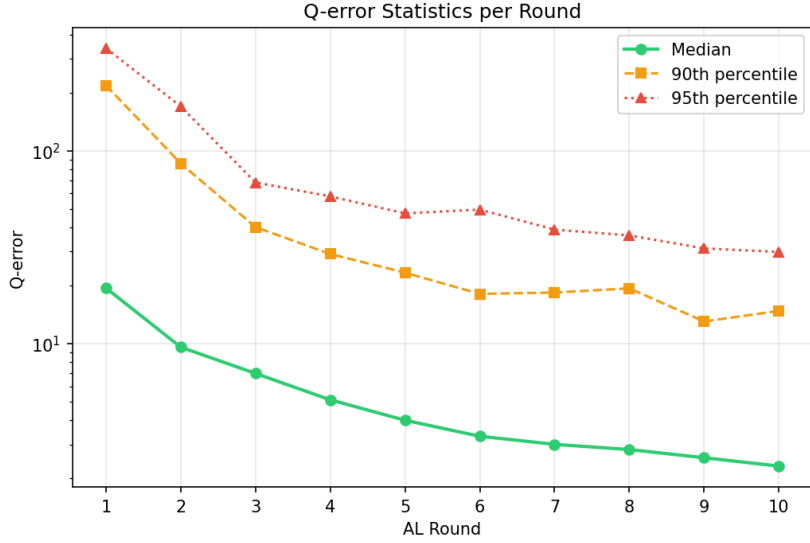


Figure 8.13: Q-error statistics across active learning rounds. The consistent decrease in all percentile metrics confirms that active query selection improves not only typical-case accuracy but also tail-end performance on the most challenging queries.

8.6 Discussion

Figure 8.18 shows the full pipeline dashboard, pulling together the key numbers from every stage into one view.

A few observations stand out. The validation pipeline rejects only a small fraction of LLM-generated queries, which confirms that the prompt constraints are doing their job. The structural distributions table counts, join counts, predicate counts are well spread rather than clustered around a single pattern, evidence that the diversity hints prevent the LLM from falling into repetitive generation.

The cardinality distribution spans several orders of magnitude, from queries returning a handful of rows to queries returning millions. That breadth is important: a model trained only on mid-range cardinalities would struggle with the extremes, and extremes are precisely where estimation errors cause the most damage to query plans.

The learning curves tell the clearest story about active learning. Accuracy improves with each round, and the improvement is front-loaded the first few rounds contribute the most because the acquisition function targets the low-hanging fruit, the queries that are easiest for the model to learn from. Later rounds bring smaller gains as the remaining unlabeled pool becomes dominated by queries the model already handles reasonably well.

The scatter plots and Q-error distributions confirm that the trained model produces well-calibrated estimates across the full range. Comparing against PostgreSQL’s native optimizer puts the learned approach in perspective: the MSCN

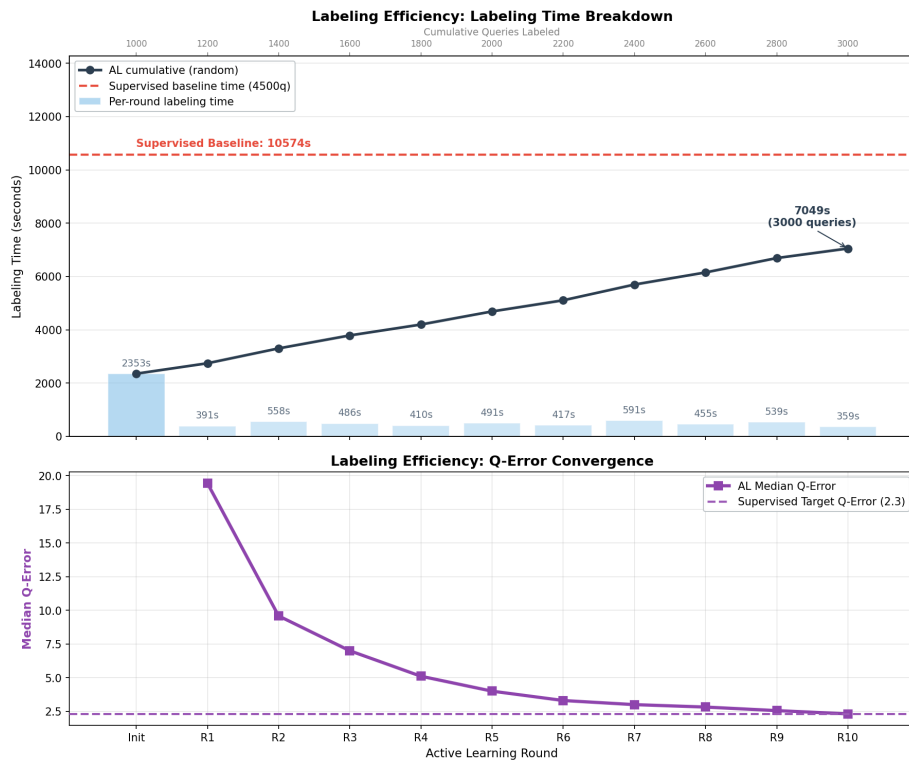


Figure 8.14: Labeling efficiency comparison. The active learning approach achieves competitive accuracy with significantly fewer labeled queries than a fully supervised baseline, demonstrating that intelligent query selection reduces the dominant cost factor in the pipeline.

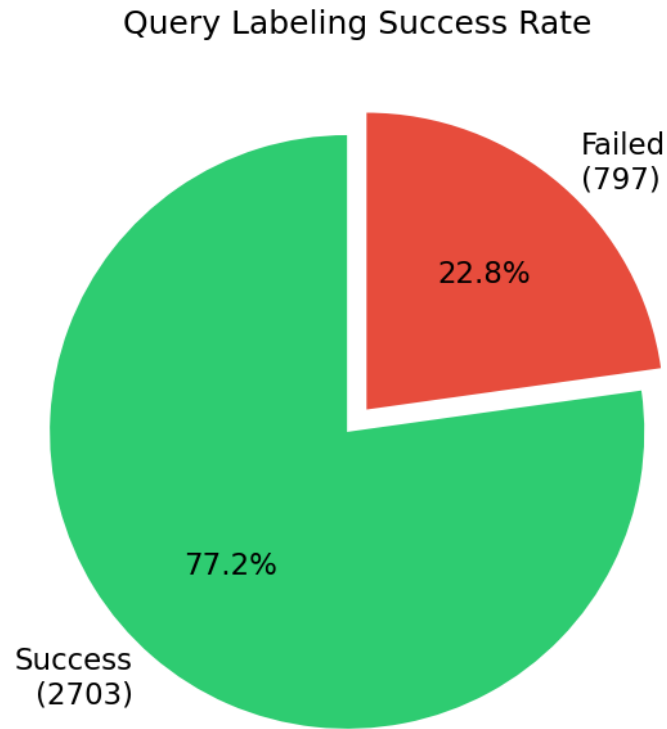


Figure 8.15: Labeling success rate across the pipeline. Queries that exceed the database timeout are assigned fallback cardinalities. The high success rate confirms that the generated queries are overwhelmingly executable within the configured time limits.

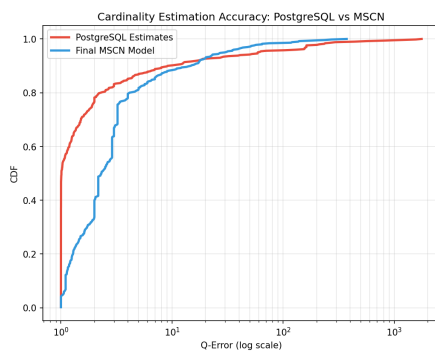


Figure 8.16: Q-error CDF comparing the MSCN model against PostgreSQL’s native estimator. A curve shifted further left indicates better estimation accuracy.

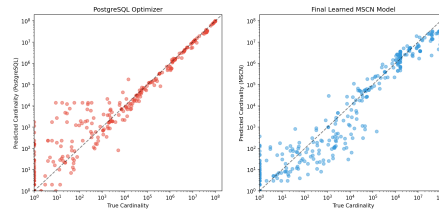


Figure 8.17: Predicted vs. actual scatter comparing MSCN and PostgreSQL. Points closer to the diagonal represent more accurate estimates.

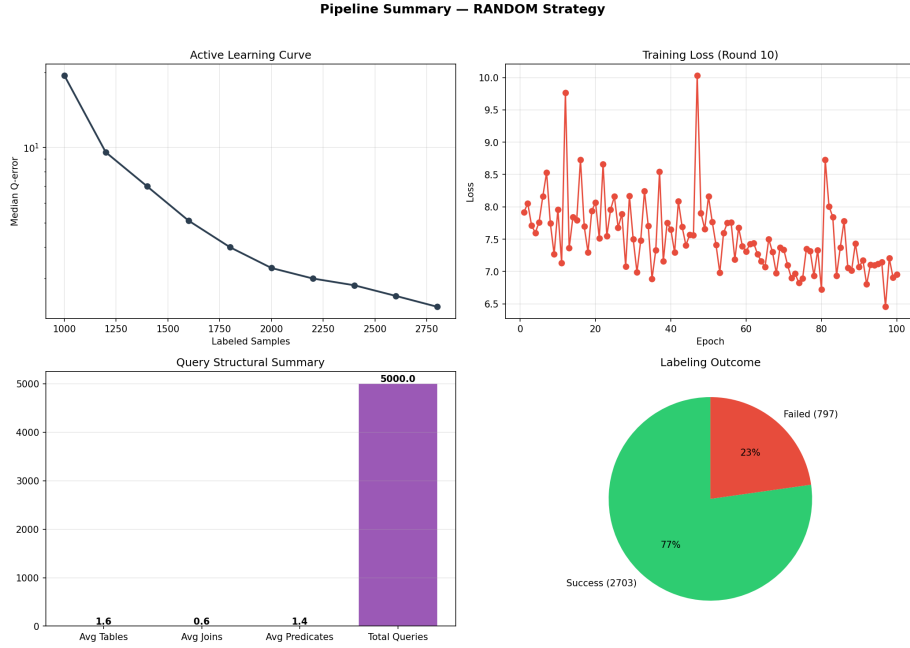


Figure 8.18: Pipeline summary dashboard. Each panel corresponds to a pipeline stage, making it possible to diagnose bottlenecks and quality issues at a glance.

model, trained on a modest number of labeled queries, can match or beat an optimizer that has access to the full table statistics and histogram infrastructure built into a production database.

Taken together, these results support the central thesis: LLM-based generation combined with active learning is a viable and efficient strategy for training learned cardinality estimators.

Chapter 9

Conclusion

9.1 What We Built and What We Learned

This thesis set out to answer a practical question: can we train a cardinality estimator well while keeping the number of expensive database executions manageable? The answer, based on our experiments, is yes. By combining LLM-based query generation with active learning, we built a pipeline that reaches competitive estimation accuracy using a fraction of the labels that a fully supervised approach would require.

Several pieces had to come together to make this work. The LLM-based generator, guided by structured prompts and diversity hints, produced workloads with enough variety in join patterns, predicate types, and selectivity profiles to give the model broad coverage of the query space. The four-layer validation filter caught the inevitable mistakes hallucinated columns, forbidden keywords, malformed syntax before they could corrupt the training data. And the active learning loop, particularly with Monte Carlo dropout as the acquisition function, proved effective at steering the labeling budget toward the queries that mattered most.

The result is a schema-agnostic, end-to-end system. Point it at a new database, give it a schema file, and it generates queries, validates them, labels the most informative ones, trains the estimator, and produces a diagnostic dashboard all without manual intervention.

9.2 Limitations

The framework as it stands has clear boundaries. The MSCN encoder supports only conjunctive queries `SELECT COUNT(*)` with AND-connected comparisons over numeric columns. Queries with OR, subqueries, `GROUP BY`, or string predicates are outside its reach. Extending the encoder to handle richer SQL fragments would require changes to both the feature extraction and the prompt constraints.

The experiments were conducted on a single benchmark dataset (IMDB). While the pipeline is schema-agnostic by design, we have not yet verified how well the trained models generalize across different data distributions or schema structures. A retail database with skewed foreign key distributions might pose challenges that IMDB does not.

Monte Carlo dropout adds computational overhead. Running twenty-five forward passes per query per round is acceptable when the alternative is executing that query on a database, but for very large unlabeled pools the scoring step can become a bottleneck. Batch-level approximations or learned acquisition functions could help.

Finally, the fallback cardinality assigned to timed-out queries introduces noise. In practice the fraction of affected queries is small, but on schemas with many expensive joins it could become significant.

9.3 Directions for Future Work

The most natural next step is extending the query fragment. Supporting disjunctions, `LIKE` predicates, and basic aggregations would make the estimator applicable to a much wider range of real workloads. This would require rethinking the encoding scheme, but the rest of the pipeline generation, validation, active learning would carry over with relatively minor modifications.

Cross-schema transfer is another promising direction. If a model trained on IMDB could be fine-tuned for a retail database using only a handful of labels, the practical value of the system would increase substantially. Meta-learning and domain adaptation techniques are worth exploring here.

On the active learning side, combining uncertainty with diversity selecting queries that are both individually uncertain and collectively cover different parts of the feature space could improve sample efficiency further. Multi-objective acquisition functions that balance exploration, exploitation, and computational cost are a natural fit.

From a systems perspective, integrating the learned estimator into a live query optimizer and measuring the end-to-end impact on query execution time would close the loop between estimation accuracy and actual system performance. The current evaluation uses Q-error as a proxy, but what ultimately matters is whether better estimates lead to better plans.

References

- [1] P. Baldi and P. Sadowski. Understanding Dropout. *Artificial Intelligence*, 2014.
- [2] A. Davitkova and S. Michel. Automated training of learned database components with generative AI. *CoRR*, abs/2512.20271, 2025.
- [3] S. Srinivasan. Efficient Monte-Carlo Dropout for Uncertainty Quantification. *SSRN*, 2024.
- [4] Y. Gal and Z. Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, volume 48 of *JMLR Workshop and Conference Proceedings*, pages 1050–1059. JMLR.org, 2016.
- [5] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, and A. Kemper. Learned cardinalities: Estimating correlated joins with deep learning. *CoRR*, abs/1809.00677, 2018.
- [6] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, and A. Kemper. Learned cardinalities: Estimating correlated joins with deep learning. In *9th Biennial Conference on Innovative Data Systems Research, CIDR 2019, Asilomar, CA, USA, January 13-16, 2019, Online Proceedings*. www.cidrdb.org, 2019.
- [7] D. Li, Z. Wang, Y. Chen, R. Jiang, W. Ding, and M. Okumura. A survey on deep active learning: Recent advances and new frontiers. *IEEE Trans. Neural Networks Learn. Syst.*, 36(4):5879–5899, 2025.
- [8] Q. Liu, B. Chen, J. Guo, M. Ziyadi, Z. Lin, W. Chen, and J. Lou. TAPEX: Table pre-training via learning a neural SQL executor. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.
- [9] S. Mansour, L. Perelli, L. Mainetti, G. Davidson, and S. D’Amato. PAARS: Persona aligned agentic retail shoppers. *CoRR*, abs/2503.24228, 2025.
- [10] P. Negi, Z. Wu, A. Kipf, N. Tatbul, R. Marcus, S. Madden, T. Kraska, and M. Alizadeh. Robust query driven cardinality estimation under changing workloads. *Proc. VLDB Endow.*, 16(6):1520–1533, 2023.
- [11] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei, G. T. Kakkar, Y. Gan, A. Saberi, F. Ozcan, and S. Ö. Arik. CHASE-SQL: Multi-path reasoning and preference optimized candidate selection in text-to-SQL. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.
- [12] A. Rahaman, A. Zheng, M. Milani, F. Chiang, and R. Pottinger. Evaluating SQL understanding in large language models. In A. Simitsis, B. Kemme, A. Queralt, O. Romero, and P. Jovanovic, editors, *Proceedings 28th International Conference on Extending Database Technology, EDBT 2025, Barcelona, Spain, March 25-28, 2025*, pages 909–921. OpenProceedings.org, 2025.

- [13] T. Schmidt, V. Leis, P. Boncz, and T. Neumann. SQLStorm: Taking database benchmarking into the LLM era. *Proc. VLDB Endow.*, 18(11):4144–4157, 2025.
- [14] X. Wang, C. Qu, W. Wu, J. Wang, and Q. Zhou. Are we ready for learned cardinality estimation? *Proc. VLDB Endow.*, 14(9):1640–1654, 2021.